

Gestao da Quinta - Evora

Data: 13 de Junho de 2026

Versao v13.0 - Auditoria de Seguranca (substitui v12.0 - este documento esmaga o anterior)

Utilizador admin: miguelseverino

REGRAS CRITICAS - LER ANTES DE QUALQUER ALTERACAO

- NUNCA usar export const dynamic = 'force-dynamic' (causa loop infinito)
- SEMPRE incluir GRANT SELECT, INSERT, UPDATE, DELETE em novas tabelas Supabase
- NUNCA substituir qualquer pagina sem pedir ficheiro atual do GitHub primeiro
- fetchRendas SEMPRE usar leasesMap separado, NUNCA join direto
- Codigo SEMPRE ficheiro completo - nunca patches parciais
- SEMPRE pedir ficheiro atual do GitHub antes de modificar qualquer pagina
- PRIMEIRA ATIVIDADE num novo chat: abrir terminal, cd C:\Projetos\quinta-app, git pull, claude
- NOVA REGRA (13/06/2026): a PRIMEIRA prioridade deste chat e a Seccao 4 (Auditoria de Seguranca) - existe uma fuga de dados ativa em producao, ver Prioridade 0

1. URLs e Acessos

App Web:

<https://quinta-gestao.vercel.app>

GitHub:

<https://github.com/mimaraalojamentos-oss/quinta-gestao>

Supabase:

<https://supabase.com/dashboard/project/szjwtccjrljprettcfxm>

Vercel:

<https://vercel.com/miguel-severino-s-projects/quinta-gestao>

SQL Editor:

<https://supabase.com/dashboard/project/szjwtccjrljprettcfxm/sql/new>

Supabase ID:

szjwtccjrljprettcfxm

Twilio Number:

+19452024418

Claude Code:

C:\Projetos\quinta-app

2. Stack Tecnica

Frontend:

Next.js 16.2.6 + TypeScript

Estilo:

Tailwind CSS v4

Base de dados:

Supabase PostgreSQL - Plano PRO - backups diarios 7 dias

Auth:

Supabase Auth - sessionStorage - timeout 15min

Alojamento:

Vercel Hobby Gratuito

IA/OCR:

@anthropic-ai/sdk 0.39.0 - Pago por uso - claude-sonnet-4-5

SMS:

Twilio +19452024418 (EUA)

Claude Code:

v2.1.170 - C:\Projetos\quinta-app - git pull antes de comecar

3. Modulos Implementados

Dashboard /dashboard

KPIs, rendas, alertas, EDP, projetos

Espacos /espacos

39 espacos

Inquilinos /inquilinos

conta corrente com eletricidade e adiantamentos

Rendas & Pagamentos /pagamentos

adiantamento automatico quando paga a mais

Eletricidade Quadros /eletricidade/quadros

3 quadros EDP, grafico atualiza automaticamente

Eletricidade Espacos /eletricidade/espacos

cobrar/acumular, filtro checkboxes por espaco

Projetos /projetos

Documentos /documentos

OCR, 7 tipos incl. transferencia_interna

Despesas /despesas

Fundo de Maneio /caixa

movimentos automaticos de documentos

Notas /notas

arquivar/restaurar

Alertas /alertas

Relatorios /relatorios

Utilizadores /utilizadores

gestao de roles + log de acessos (so admin)

4. AUDITORIA DE SEGURANCA (13/06/2026) - PRIORIDADES PARA ESTE CHAT

Auditoria realizada em 13/06/2026. Conclusao: existe uma fuga de dados ATIVA e exploravel sem qualquer login. As prioridades abaixo estao ordenadas - resolver pela ordem indicada, comecando pela PRIORIDADE 0 que e a mais urgente de todo o projeto.

Resumo rapido:

0 - Fuga de dados sem login (RLS/anon)	CRITICO MAXIMO - ATIVO AGORA
1 - /api/create-user + /api/update-user sem role	CRITICO
2 - RLS "authenticated = tudo" nas tabelas core	CRITICO
3 - Tabela profiles sem policy versionada	CRITICO
4 - process-document/process-contract sem role	IMPORTANTE
5 - /api/send-sms aberto + sent_by falso	IMPORTANTE
6 - xlsx 0.18.5 vulnerabilidade alta sem fix	IMPORTANTE
R - Headers HTTP, postcss, schemas em falta	RECOMENDACAO

PRIORIDADE 0 [CRITICO MAXIMO - ATIVO AGORA] - Fuga de dados sem autenticacao

Testado em 13/06/2026 com a chave publica NEXT_PUBLIC_SUPABASE_ANON_KEY (role: anon), SEM qualquer login - exatamente o que um visitante anonimo consegue fazer a partir da consola do browser (a anon key vai embutida no JS publico do site). As seguintes tabelas devolveram DADOS/CONTAGENS REAIS a um utilizador anonimo (GET ?select=id&limit=1 + Prefer: count=exact):

tenants	-> 32 linhas
rent_payments	-> 180 linhas
profiles	-> 4 linhas
cash_fund_movements	-> 43 linhas
expenses	-> 219 linhas
leases	-> 32 linhas
spaces	-> 39 linhas
bank_transactions	-> 146 linhas
banks	-> 1 linha
sms_log	-> 3 linhas
gate_phones	-> 69 linhas
electricity_readings	-> 54 linhas
electricity_charges	-> 3 linhas

As seguintes devolveram 0 linhas - podem estar vazias OU protegidas, nao ha forma de distinguir so com leitura: documents, access_logs, debts, debt_payments, owners, tenant_contacts, notes, meters, meter_readings.

Causa provavel: RLS nao esta ativo nestas tabelas em producao (ou existe policy "TO public"/"anon"), apesar de supabase-schema.sql definir "FOR ALL TO authenticated USING (true)".

ACAO IMEDIATA (fazer ANTES de qualquer outra alteracao):

- 1. Abrir Supabase Dashboard -> Database -> Tables -> verificar se "RLS enabled" esta ativo em TODAS as tabelas do schema public (as 13 listadas + as 9 que devolveram 0).
- 2. Para cada tabela sem RLS: ALTER TABLE <tabela> ENABLE ROW LEVEL SECURITY;
- 3. Ir a Database -> Policies e procurar policies "TO public" ou que incluam "anon" - remover/corrigir.
- 4. Recriar policies para excluir anon por completo e introduzir controlo por role (ver Prioridade 2).
- 5. Verificar especificamente a tabela profiles: confirmar se UPDATE de "role" e permitido a qualquer utilizador (mesmo anonimo). Se sim, e a forma mais direta de account takeover total: PATCH profiles SET role=admin sem login.
- 6. Depois de corrigir, correr: node scripts/rls-check.mjs (existe no repo, le .env.local, faz so GET com limit=1 usando a anon key, nao escreve nada). Sucesso = todas as tabelas devolvem status 401 OU rows_returned=0 / content-range=*/0 para o role anon.

NOTA: nao foram testados INSERT/UPDATE/DELETE com a anon key (seria destrutivo em producao). Assumir que a escrita tambem esta exposta enquanto o RLS nao for corrigido.

PRIORIDADE 1 [CRITICO] - /api/create-user e /api/update-user sem verificacao de role

Ficheiros: app/api/create-user/route.ts, app/api/update-user/route.ts

Estas rotas usam SUPABASE_SERVICE_ROLE_KEY (ignora RLS) e NAO verificam profiles.role do utilizador que faz o pedido - middleware.ts so garante "esta autenticado", nao distingue admin de viewer.

Com qualquer conta autenticada (incl. "Visualizador"):

- GET /api/create-user -> lista id + email de todos os utilizadores (incl. admins)
- POST /api/create-user {email,password,role:'admin'} -> cria conta admin nova (escalamento total)
- POST /api/update-user {userId:<id-admin>, password:'nova'} -> rouba conta admin (account takeover)
- DELETE /api/create-user {id:<id-admin>} -> apaga conta admin (sabotagem)

ACAO: no inicio de cada handler, obter a sessao via lib/supabase-server.ts (createClient), buscar profiles.role do utilizador autenticado e devolver 403 se role !== "admin".

PRIORIDADE 2 [CRITICO] - RLS "authenticated = tudo" em todas as tabelas core

Ficheiro: supabase-schema.sql (linhas ~110-127)

Todas as tabelas (spaces, tenants, leases, rent_payments, electricity_readings, electricity_charges, expenses, cash_fund_movements) tem:

```
CREATE POLICY "Authenticated users can do everything" ON <tabela>
  FOR ALL TO authenticated USING (true) WITH CHECK (true);
```

O controlo de role (isAdmin, isCoAdmin, isElectrician em lib/auth-context.tsx) e so cosmetico/UI. Um "Visualizador" autenticado pode abrir a consola do browser e correr supabase.from("expenses").delete()... e a base de dados aceita.

ACAO: substituir por policies role-based que leem profiles.role:

- viewer -> apenas SELECT
- electrician -> SELECT em tudo + INSERT/UPDATE nas tabelas de eletricidade
- admin/coadmin -> CRUD total

Seguir o padrao ja correto em supabase-access-logs.sql.

PRIORIDADE 3 [CRITICO] - Tabela profiles sem schema/policy versionados

profiles guarda o role de cada utilizador mas NAO tem CREATE TABLE nem policy em supabase-schema.sql nem supabase-access-logs.sql. app/perfil/page.tsx e app/utilizadores/page.tsx fazem supabase.from("profiles").update(...) direto do browser.

ACAO: confirmar na Supabase Dashboard a policy de UPDATE em profiles - deve permitir ao proprio utilizador so atualizar "name" (NUNCA "role"); "role" so editavel via policy/funcao restrita a admin. Versionar essa policy num novo ficheiro .sql.

PRIORIDADE 4 [IMPORTANTE] - /api/process-document e /api/process-contract sem check de role

Ficheiros: app/api/process-document/route.ts, app/api/process-contract/route.ts

Protegidos so pelo middleware (= "esta autenticado"). Qualquer utilizador (incl. Visualizador/Eletricista) pode chamar estas rotas, consumindo a API Anthropic (custo, claude-sonnet-4-5) e criando documents/expenses/cash_fund_movements/meter_readings - contorna a regra "Visualizador = so leitura".

ACAO: adicionar o mesmo check de role da Prioridade 1 (bloquear viewer; provavelmente bloquear electrician tambem em process-contract).

PRIORIDADE 5 [IMPORTANTE] - /api/send-sms aberto + log "sent_by" falso

Ficheiro: app/api/send-sms/route.ts

- Qualquer utilizador autenticado consegue enviar SMS arbitrarios (mensagem + lista de numeros) via Twilio do projeto (+19452024418) - risco de custo / SMS bombing.
- Linha ~67: sent_by: "admin" esta FIXO no codigo, independente de quem enviou - sms_log fica com auditoria errada.

ACAO: restringir a admin/coadmin, validar recipients contra gate_phones, e gravar o user_id/email real do utilizador autenticado (padrao logAccess ja existe no projeto).

PRIORIDADE 6 [IMPORTANTE] - Dependencia xlsx@0.18.5 com vulnerabilidade alta

npm audit reporta xlsx - Prototype Pollution + ReDoS, severidade alta, sem fix disponivel no npm.

ACAO: verificar onde xlsx e usado (exportacao vs. importacao de ficheiros enviados por utilizadores). Se importa ficheiros externos, e vetor de DoS/poluicao de prototipo - considerar SheetJS oficial via CDN (cdn.sheetjs.com) ou remover se nao for essencial.

RECOMENDACOES (baixa prioridade, fazer depois das anteriores)

- .env.local e segredos: corretos, fora do git (confirmado via git ls-files e git log --all) - sem accao.
- next.config.ts sem headers de seguranca HTTP (CSP, X-Frame-Options, Referrer-Policy, HSTS, X-Content-Type-Options) - adicionar via headers().
- postcss < 8.5.10 dentro de node_modules/next - severidade moderada, so build-time, acompanhar updates do Next.
- Versionar schema + polices das tabelas que faltam em supabase-schema.sql: profiles, gate_phones, sms_log, debts, debt_payments, owners, tenant_contacts, notes, meters, meter_readings, banks, bank_transactions, electricity_config, dismissed_alerts, projects.

Ferramenta de verificacao

scripts/rls-check.mjs - script Node (so usa fetch nativo, le .env.local) que faz GET ?select=id&limit=1 com header "Prefer: count=exact" contra cada tabela, usando a anon key (role anon, sem login). Nao escreve nada na base de dados. Editar o array "tables" no topo do ficheiro conforme necessario. Correr com:

```
node scripts/rls-check.mjs
```

Resultado esperado APOS corrigir RLS: status 401 (ou 200 com rows_returned=0 e content-range=*/0) para todas as tabelas, quando testado com a anon key.

Plano de acao sugerido para este chat

- 1. Prioridade 0 - corrigir RLS na Supabase Dashboard (URGENTE - fuga de dados ativa, fazer primeiro)
- 2. Prioridade 1 - blindar /api/create-user e /api/update-user com check de role admin
- 3. Prioridades 2 e 3 - reescrever polices RLS role-based (incl. profiles), versionar em .sql
- 4. Prioridades 4 e 5 - adicionar checks de role em process-document, process-contract, send-sms
- 5. Prioridade 6 e Recomendacoes - resolver xlsx, adicionar headers HTTP, versionar schemas em falta